

Standard EHR & HL7 FHIR — Architecture Proposal (Team D)

Module: Standard EHR / HL7 FHIR R4 Interoperability Layer **Pillar:** 4 of 4 (Security, Agentic AI, Dashboard, Standard EHR) **Architecture Tier:** Architecture B — Recommended **Core Stack:** Hyperledger Fabric v2.x · IPFS · CouchDB · Node.js/Express · HL7 FHIR R4

1. System Overview & Scope

Team D owns the **Standard EHR / FHIR layer**: the boundary that converts every clinical artifact entering the platform — lab reports, encounters, prescriptions, consent grants — into HL7 FHIR R4 resources before anything is hashed, encrypted, stored, or anchored.

The module sits directly beneath the Security layer and directly above Storage in the platform’s four-pillar model:



Without a shared data model, the Agentic AI team’s agents would each need to learn a different hospital’s JSON dialect, the Security team’s VCAP hashes would not be reproducible across organizations, and the Dashboard team could not render a uniform Agent Trace. FHIR is what lets the other three pillars talk about the “same patient record” without agreeing on anything else.

Zero-PHI off-chain principle (binding on this module): Team D’s adapters and validators never write a raw FHIR payload, a clinical narrative, or any PHI field to the Fabric ledger. Fabric only ever receives a SHA-256 digest of a canonicalized FHIR payload, an IPFS CID, and non-clinical status/state attributes. This constraint is enforced at the schema level (Section 7) and the integration level (Section 9), not left to convention.

2. Current System Analysis

2.1 What exists today

The project currently represents clinical data as ad hoc, per-service JSON, e.g.:

```
{ "patientId": "P001", "hb": 8.5 }
```

There is no shared resource model between the lab gateway, the hospital record store, and the AI pipeline — each service defines its own shape and its own field names for the same concept (a hemoglobin value).

2.2 Existing gateway surface (audited)

Component	Role today	Disposition
report-extractor.js	OCR/text extraction from uploaded lab PDFs into custom JSON	Keep as upstream producer; Team D adapts its output, does not rewrite it
server.js	API/orchestration entry point (/api/ai/analyze , /api/records/ipfs)	Keep ; Team D adds /api/fhir/* routes alongside it
ai-agent.js	Gemini-based lab analysis + CDS agent (Agentic AI team)	Unchanged by Team D ; consumes FHIR Observation once available
ipfs.js	Off-chain artifact storage	Reused ; Team D’s bundler calls the existing client rather than reimplementing IPFS pinning
Fabric chaincode (accessContract.js , lab-result state)	On-chain metadata + consent	Extended , not replaced — FHIR adds patientRecordHash and resource-type fields to existing state schema

2.3 Existing storage mechanics

Large artifacts already go to IPFS with only a CID and metadata anchored on Fabric — this zero-PHI-on-chain pattern already exists for lab PDFs and is **carried forward unchanged** for FHIR payloads; Team D’s job is standardizing what goes into that envelope, not the envelope mechanism itself.

2.4 What should NOT be rewritten

- `report-extractor.js` OCR/extraction logic
- `ipfs.js` pinning/retrieval client
- The existing `/api/ai/analyze` and `/api/records/ipfs` endpoints (kept for backward compatibility during migration)
- The Gemini-based analysis agents themselves (they gain a FHIR-shaped input, not a new implementation)

3. Problem & Gap Analysis

3.1 Interoperability barriers

Custom JSON means every consumer (AI agents, dashboard, cross-org partners) needs a bespoke parser, and there is no way for an external hospital system to exchange data with this platform without a private integration.

3.2 Risks of custom schemas

Risk	Consequence
No terminology binding	<code>"hb": 8.5</code> has no unit, no code — ambiguous across labs (g/dL vs mmol/L)
No canonical serialization	Two services with the same clinical facts produce different hashes, breaking VCAP’s tamper-detection guarantee
Field-name drift	<code>patientId</code> vs <code>patient_id</code> vs <code>pid</code> across services silently breaks joins
No resource versioning	Amendments to a diagnostic report have no defined history model

3.3 Deterministic hashing challenge

VCAP (owned by Security, Teams 2/5) requires a `patientRecordHash` that is reproducible by any party re-computing it from the same clinical facts. Plain `JSON.stringify()` is **not** deterministic across key ordering, and Section 17.2 of the Security team’s specification already flags this: identical content re-serialized with reordered keys must not silently produce a different SHA-256 digest. This is the single hardest requirement driving this proposal’s design.

3.4 Lack of terminology standardization

Free-text test names (“Hb”, “Hemoglobin”, “HGB”) cannot be safely compared or trended by the Agentic AI team’s Patient History Agent without a controlled vocabulary.

4. Requirements

4.1 Functional

- **FR-1** Ingest legacy custom-JSON clinical events and map them to 7 target FHIR R4 resources.
- **FR-2** Validate every resource against its FHIR profile and required terminology bindings before it leaves the adapter.
- **FR-3** Canonicalize every payload under RFC 8785 (JCS) prior to hashing.
- **FR-4** Encrypt canonicalized payloads client-side with AES-256-GCM and package for IPFS.
- **FR-5** Emit `patientRecordHash` (SHA-256 over the JCS-canonical form) to the Security team’s VCAP pipeline and to the on-chain metadata record.
- **FR-6** Expose a stable adapter/tool interface (`getFHIRResource` , `createFHIRResource`) consumable by the Agentic AI orchestrator’s tool layer.

4.2 Non-Functional

Target	Value
Schema + terminology validation latency (p95)	< 40 ms — matches the Security team’s ingestion latency budget (Section 25.1 of their spec)
Canonicalization determinism	100% — identical clinical content → identical digest, independent of source-JSON key order
Storage efficiency	Only hash + CID on-chain; full FHIR bundle off-chain in IPFS
Adapter throughput	Sustain the Lab Gateway’s existing ingestion rate with no added queueing under normal load

4.3 Security & Compliance

- **Zero-PHI on-chain enforcement:** the chaincode schema (Section 7.1) has no field capable of holding a clinical narrative or PHI value — this is a schema-level guarantee, not a code-review convention.
- **Crypto-shredding compatibility:** because FHIR payloads live only in IPFS ciphertext (per the Security team’s envelope-encryption model), Team D’s on-chain records remain non-identifying after a Security-team-executed key destruction, satisfying GDPR Art. 17 the same way the rest of the platform does.
- **Regulatory alignment:** resource choices and terminology bindings track HIPAA identifiers, GDPR data-minimization, and ABDM/IPS profile expectations (Section 6).

5. Proposed Architecture (Architecture B — Recommended)



Every arrow above is a hard boundary: nothing left of the AES-256-GCM stage is ever persisted; nothing right of the canonicalizer is ever un-hashed on-chain.

6. HL7 FHIR Standardization & Profiles

6.1 Target resource mapping

Legacy concept	FHIR R4 Resource	Notes
Patient record	Patient	Identifier = patientIdHash reference, not raw MRN
Lab measurement	Observation	LOINC-coded, UCUM units
Lab report	DiagnosticReport	References supporting observation s
Prescription	MedicationRequest	RxNorm/SNOMED-coded when available
Visit	Encounter	Links Patient , Practitioner , Organization
Diagnosis	Condition	SNOMED-CT / ICD-10 coded
Access grant	Consent	Maps to accessContract.js composite key (Section 9)

6.2 Terminology bindings

Concept	System	Example
CBC panel test codes	LOINC	718-7 = Hemoglobin
Lab result units	UCUM	g/dL , 10*3/uL
Diagnoses	SNOMED-CT / ICD-10	Condition.code
National profile alignment	ABDM (India) / IPS (International Patient Summary)	Patient identifier system, DocumentReference profile

6.3 Example mapping (legacy → FHIR)

```
// Before (legacy)
{ "patientId": "PAT001", "test": "CBC", "hb": 8.5 }

// After (FHIR Observation)
{
  "resourceType": "Observation",
  "id": "OBS-CBC-HB-0001",
  "status": "final",
  "subject": { "reference": "Patient/PAT001" },
  "code": {
    "coding": [
      { "system": "http://loinc.org", "code": "718-7", "display": "Hemoglobin" }
    ]
  },
  "valueQuantity": { "value": 8.5, "unit": "g/dL", "system": "http://unitsofmeasure.org", "code": "g/dL" }
}
```

7. Data Model & Schemas

7.1 Minimal Fabric On-Chain CouchDB Index Schema

```
{
  "recordId": "RECORD~<patientIdHash>~<recordId>",
  "resourceType": "DiagnosticReport",
  "patientIdHash": "hmac-sha256:...",
  "patientRecordHash": "sha256:...",
  "dataCID": "bafybeic7x42mpn6u7iynr4l4qgsqd5j26wf5j2l3",
  "status": "FINAL",
  "canonicalizationScheme": "RFC8785-JCS",
  "timestamp": "2026-09-16T10:30:00Z"
}
```

No clinical value, code, or narrative field exists in this schema — by construction, a full ledger copy at any peer organization reveals only hashes, CIDs, and status. This mirrors the Security team’s zero-PHI state-key design (`RECORD~<patientIdHash>~<recordId>`) exactly, so Team D’s records slot into their existing chaincode namespace rather than introducing a parallel one.

7.2 Off-Chain FHIR DiagnosticReport / Observation Payload Schema

```
{
  "resourceType": "DiagnosticReport",
  "id": "DR-0001",
  "status": "final",
  "subject": { "reference": "Patient/PAT001" },
  "result": [{ "reference": "Observation/OBS-CBC-HB-0001" }],
  "issued": "2026-09-16T10:30:00Z",
  "performer": [{ "reference": "Organization/DiagnosticsOrg" }],
  "contained": [
    {
      "resourceType": "Observation",
      "id": "OBS-CBC-HB-0001",
      "status": "final",
      "code": { "coding": [{ "system": "http://loinc.org", "code": "718-7" }] },
      "valueQuantity": { "value": 8.5, "unit": "g/dL", "system": "http://unitsofmeasure.org", "code": "g/dL" }
    }
  ]
}
```

This is the object that is canonicalized, hashed, encrypted, and pinned — never the on-chain record.

7.3 Canonicalization rule set (RFC 8785 / JCS)

1. Object keys sorted lexicographically (byte-wise on UTF-8).
2. No insignificant whitespace.
3. Numbers normalized per the ECMAScript `Number.toString()` grammar JCS specifies (no leading zeros, no trailing `.0`).
4. Strings in normalized UTF-8, no unnecessary escaping.
5. Canonical form is hashed with SHA-256 to produce `patientRecordHash` — the exact input the Security team's VCAP tuple $(H(R), H(P), F(M), H(S), t, \sigma)$ expects for $H(R)$.

8. Feature Specification

Feature 1 — Dynamic Legacy-to-FHIR Adapter Engine Declarative, per-source mapping rules (JSON path → FHIR path) so a new upstream format (a new lab's export, say) is onboarded by adding a mapping file, not by writing new adapter code.

Feature 2 — High-Performance Schema & Terminology Validator Validates structural conformance (`@asymmetrik/fhir-json-schema-validator`) and terminology membership (LOINC/UCUM/SNOMED/ICD-10 lookups) in a single pass, target < 40 ms p95.

Feature 3 — RFC 8785 JCS Canonicalizer & VCAP Hash Generator Produces the canonical byte form and the resulting SHA-256 `patientRecordHash`, exposed as a library function both the ingestion adapter and the independent verification path (Security team's `verifyVCAP`) can call, so both sides compute the digest the same way.

Feature 4 — Client-Side IPFS Encrypted Bundler Wraps the existing `ipfs.js` client: takes canonicalized JSON, hands it to the Security team's AES-256-GCM envelope-encryption routine (Section 17.3 of their spec — DEK wrapped under a patient-scoped KEK), and pins the ciphertext, returning the CID for anchoring.

9. Cross-Module Integration Interfaces

9.1 Security Team (Teams 2 & 5)

- `Consent` FHIR resources map 1:1 to `accessContract.js` composite keys (`CONSENT~<patientIdHash>~<granteeId>~<scope>`), so a FHIR-level consent grant and a chaincode-level consent grant are the same fact represented twice, never two sources of truth.
- Team D supplies `patientRecordHash = H(R)` for the VCAP tuple $(H(R), H(P), F(M), H(S), t, \sigma)$; Team D does not compute $H(P)$, $F(M)$, $H(S)$, or σ — those remain the Security/AI teams' responsibility.
- Patient identifiers in FHIR `Patient.identifier` are passed through the Security team's keyed `HMAC-SHA256(Knet, patientId)` construction before ever reaching a state key — Team D never writes a plain-hashed or plaintext identifier to Fabric.

9.2 Agentic AI Team (Teams 3 & 6)

- Team D exposes two tool-layer functions matching the Agentic AI team’s controlled tool interface: `getFHIRResource(resourceType, id)` and `createFHIRResource(resourceType, payload)`. These are called by agents through their tool-validation/authorization layer — no agent talks to the adapter or the Fabric ledger directly.
- The Lab Report Agent receives structured `Observation` resources instead of raw OCR text, removing the need for each agent to parse free-text lab reports itself.
- Structured AI outputs (summaries, risk flags) that Team D receives back are treated as assistive annotations, not new clinical facts — they do not get re-ingested as FHIR resources unless a clinician-reviewed workflow explicitly promotes them (out of scope for this module).

9.3 Dashboard Team (Team 1)

- Team D emits a `FHIR_CONVERSION_STARTED` / `FHIR_CONVERSION_COMPLETED` / `FHIR_CONVERSION_FAILED` event, using the platform’s common event envelope (`eventId`, `eventType`, `timestamp`, `source`, `peerId`, `severity`, `payload`), corresponding to **Step 3 (“FHIR conversion”)** of the Dashboard team’s Agent Trace UI pipeline:

```
Report received -> OCR/Parsing -> [FHIR conversion] -> Agent processing
                  -> Governance/VCAP -> IPFS -> Blockchain anchor
```

- Event payload includes `recordId`, `resourceType`, `validationStatus`, `durationMs`, and (on success) `patientRecordHash` and `dataCID` — enough for the dashboard’s per-execution trace to show stage timestamp, duration, status, and the on-chain/CID reference without exposing any clinical value.
- These events are read by the Dashboard backend’s real-time layer (WebSocket/SSE) and surfaced under `GET /api/dashboard/agents/:agentId/executions`; Team D does not build or own that API, it only emits the event.

10. Technology Choices & Justifications

Layer	Choice	Justification	Rejected alternative	Why rejected
Runtime	Node.js / Express	Matches existing gateway stack (<code>server.js</code> , <code>gateway.js</code>)	Python/FastAPI	Would split the gateway across two runtimes for no functional gain
FHIR schema validation	@asymmetrik/fhir-json-schema-validator (+ <code>ajv</code>)	Mature JSON-Schema-based FHIR R4 validation, fast enough for the 40 ms budget	Full HL7 reference validator (Java)	Adds a JVM dependency and cross-process latency to a hot ingestion path
Canonicalization	<code>canonicalize</code> npm package (RFC 8785)	Direct, spec-conformant JCS implementation	Hand-rolled key-sort + <code>JSON.stringify</code>	Easy to get subtly wrong on number formatting; RFC 8785 exists precisely to avoid that
IPFS client	Existing <code>ipfs.js</code> (IPFS HTTP Client)	Already integrated and operational; reuse avoids duplicate pinning logic	New IPFS SDK	No functional benefit, adds migration risk
Terminology data	Bundled LOINC/UCUM/SNOMED-CT subset for the 7 target resources	Keeps validator self-contained and within the latency budget	Live external terminology server (FHIR <code>\$validate-code</code>)	Network round-trip per validation breaks the 40 ms target; deferred to Architecture C

11. Failure Handling

Failure	Behavior
Schema validation failure	Reject at the adapter; record returns <code>SCHEMA_INVALID</code> with the failing FHIR path; nothing proceeds to canonicalization
Unrecognized terminology / unit	Reject with <code>TERMINOLOGY_UNBOUND</code> ; do not silently pass through an uncoded value
Canonicalization mismatch (recomputed hash $\neq$ expected)	Treat as <code>CANONICALIZATION_ERROR</code> ; block anchoring — this must never silently produce a divergent <code>patientRecordHash</code> , since that would defeat VCAP tamper-detection
IPFS node timeout	Retry with backoff (bounded); on exhaustion, mark <code>IPFS_UNAVAILABLE</code> and do not anchor a CID that was never actually pinned

12. Scalability & Future Extensions

- **Batch FHIR ingestion:** bulk-import endpoint for migrating existing custom-JSON records in bulk rather than one-by-one.
- **Validated-schema caching:** cache compiled JSON-Schema validators per resource type to avoid recompilation cost on every request.
- **Architecture C (future/research tier):** Native FHIR Search API (`GET /fhir/Observation?patient=...&code=...`) and a live terminology server for `$validate-code / $expand` operations, once the 40 ms latency budget can absorb a network hop (e.g., via a co-located terminology cache).

13. Implementation Plan & Roadmap

Phase	Scope
1 — Schemas & Terminology	Define the 7 FHIR profiles; bundle LOINC/UCUM/SNOMED-CT/ICD-10 subsets; write example payloads
2 — Validation & JCS	Implement the schema validator and the RFC 8785 canonicalizer + hash generator; unit-test determinism
3 — Gateways / Adapters	Build the legacy-to-FHIR adapter for <code>report-extractor.js</code> output; wire the IPFS bundler and Fabric anchoring
4 — Cross-Module Integration	Wire <code>getFHIRResource / createFHIRResource</code> for the Agentic AI tool layer; wire VCAP hash hand-off to Security; wire <code>FHIR_CONVERSION_*</code> events to the Dashboard

14. Testing Plan

Unit tests - Schema validation: valid resource passes; each required-field omission fails with the correct code. - JCS determinism: two structurally-identical objects with different key order produce the identical canonical string and identical SHA-256 digest.

Negative tests - Malformed FHIR resource (missing `resourceType`) → rejected before canonicalization. - Unauthorized/unrecognized terminology code → rejected with `TERMINOLOGY_UNBOUND` . - Re-ordered JSON keys must not change `patientRecordHash` (regression guard for Section 3.3's core risk).

End-to-end integration tests - Full pipeline: legacy JSON in → FHIR `DiagnosticReport` out → canonical hash → IPFS CID → Fabric anchor → dashboard `FHIR_CONVERSION_COMPLETED` event observed. - VCAP round-trip: hash produced by Team D's canonicalizer is independently recomputed by the Security team's `verifyVCAP` path and matches.

15. Deliverables & File Structure

```
fhir/
├─ schemas/
│  ├─ patient.schema.json
│  ├─ observation.schema.json
│  ├─ diagnostic-report.schema.json
│  ├─ medication-request.schema.json
│  ├─ encounter.schema.json
│  ├─ condition.schema.json
│  └─ consent.schema.json
├─ terminology/
│  ├─ loinc-subset.json
│  ├─ ucum-units.json
│  ├─ snomed-subset.json
│  └─ icd10-subset.json
├─ src/
│  ├─ adapters/
│  │  └─ legacy-to-fhir.js
│  ├─ validator.js
│  ├─ canonicalizer.js      # RFC 8785 (JCS)
│  ├─ hasher.js             # patientRecordHash generator
│  ├─ ipfs-bundler.js
│  ├─ fabric-anchor.js
│  └─ tools/
│     ├─ getFHIRResource.js
│     └─ createFHIRResource.js
├─ examples/
│  ├─ observation-cbc.json
│  └─ diagnostic-report-lab.json
├─ resource-mapping.md
├─ ehr-schema.md
├─ validation-design.md
├─ blockchain-storage-model.md
├─ ipfs-storage-model.md
├─ migration-plan.md
└─ implementation-roadmap.md
```